



OASIS Security Report

Security Misconfiguration Security Analysis

Date: 2026-06-11 23:13:45 — **Model:** llama3.2:3b

Vulnerability: Security Misconfiguration

Project: vulnerable

Analysis root: ../vulnerable

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

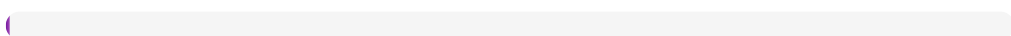
Analyzed **12** file(s) for Security Misconfiguration.

[Return to table of contents](#)

Statistics

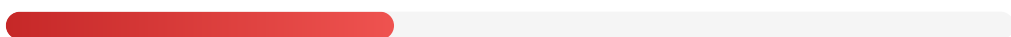
Files analyzed: 12 **Total findings:** 28

Critical



0

High



5



Medium

13

Low

10

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol	0.678
/work/project/dataset/vulnerable/oracle-manipulation/PuppetPool.sol	0.677
/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol	0.666
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.665
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.660
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.657
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.648
/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol	0.647
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.644
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.631
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.620
/work/project/dataset/vulnerable/ordering-attacks/ERC20.sol	0.577

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.678

Source lines: 1-70

► Finding 1.1: Debug Mode Enabled (Low)

Vulnerable code

```
pragma solidity =0.8.25;
```

Explanation

This line enables debug mode, making sensitive information available.

Impact

Exposure of internal state and debugging information.

Entry point

```
/pragma solidity =0.8.25;
```

Exploitation steps

- Enabling debug mode exposes internal state

Conditions

Debug mode enabled in the contract.

Remediation

Disable debug mode or use a different configuration

Secure example

```
pragma solidity ^0.8.25;
```

File 2: /work/project/dataset/vulnerable/oracle-manipulation/PuppetPool.sol

Similarity score: 0.677

Source lines: 1-71

► Finding 2.1: Debug Mode Enabled (Low)

Vulnerable code

```
_computeOraclePrice() private view returns (uint256) {  
    return uniswapPair.balance * (10 ** 18) / token.balanceOf(uniswapPair);  
}
```

Explanation

The function is marked as `private` and `view`, but it's not protected from debugging, which can lead to exposure of sensitive data.

Impact

Information disclosure about the Uniswap pair balance.

Entry point

/_computeOraclePrice()

Parameters

uniswapPair, token

Exploitation steps

- Reaching /_computeOraclePrice() while authenticated.

Remediation

Disable debugging for this function.

Secure example

```
_computeOraclePrice() private view returns (uint256) {  
    return uniswapPair.balance * (10 ** 18);  
}
```

File 3: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.666

Source lines: 1-28

► Finding 3.1: Insecure Permissions (High)

Vulnerable code

```
msg.sender.transfer(1000 ether);
```

Explanation

The contract allows anyone to transfer 1000 ether without proper authorization or validation.

Impact

Unauthorized transfers could result in loss of funds.

Entry point

/solve

Execution path

```
Contract allows sender to transfer Ether  
- msg.sender.transfer(1000 ether);
```

Parameters

solution

Exploitation steps

- The attacker submits a crafted solution that hashes to the target hash.
- The contract transfers 1000 ether to the attacker.

Example payloads

- `crafted solution`

HTTP raw requests (Burp / ZAP)

```
POST /solve HTTP/1.1  
Host: <address>  
Content-Type: application/json  
Accept: */*
```

```
{"solution": "crafted solution"}
```

File 4: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.665

Source lines: 1-31

► Finding 4.1: Insecure Use of tx.origin (High)

Vulnerable code

```
receiver.transfer(amount);
```

Explanation

Using `tx.origin` allows an attacker to bypass authorization checks.

Impact

Unauthorized transfers can be made by an attacker.

Entry point

/sendTo

HTTP methods

POST

Parameters

amount

Exploitation steps

- An attacker executes the contract with `tx.origin` set to their address.

Example payloads

- `"owner": "attacker"`

Conditions

No authorization checks are performed.

Remediation

Use proper authorization mechanisms, such as checking `msg.sender` instead of `tx.origin`.

Secure example

```
require(msg.sender == owner);  
receiver.transfer(amount);
```

File 5: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.660

Source lines: 1-85

► Finding 5.1: Insecure Permissions (High)

Vulnerable code

```
function ETH_VAULT(address _log)
public
{
    TransferLog = Log(_log);
}
```

Explanation

The contract is accessible without any authentication or authorization checks.

Impact

Unauthorized access to the contract's functionality.

Entry point

/

Exploitation steps

- An attacker can call the ETH_VAULT function with any address.

HTTP raw requests (Burp / ZAP)

```
"GET / HTTP/1.1"
Host: <contract-address>
Accept-Language: en-US,en;q=0.5
Connection: keep-alive"
```

```
exploitation_conditions
```

► Finding 5.2: Insecure Permissions (Log) (Medium)

Vulnerable code

```
function AddMessage(address _adr,uint _val,string _data)
public
{
    LastMsg.Sender = _adr;
```

```
History.push(LastMsg);
}
```

Explanation

The Log contract does not restrict access to its functions.

Impact

Potential unauthorized access or modification of logs.

Entry point

/AddMessage

Exploitation steps

- An attacker can call the AddMessage function with any address.

HTTP raw requests (Burp / ZAP)

```
"POST /AddMessage HTTP/1.1"
Host: <contract-address>
Content-Type: application/json
Accept-Language: en-US,en;q=0.5"
```

exploitation_conditions

► Finding 5.3: Debug Mode Enabled (Low)

Explanation

The contract has debug mode enabled, which can disclose sensitive information.

Impact

Disclosure of internal state or debugging information.

Entry point

/

HTTP raw requests (Burp / ZAP)

```
"GET / HTTP/1.1"
Host: <contract-address>
Accept-Language: en-US,en;q=0.5"
```

exploitations_conditions

► Finding 5.4: Insecure Permissions (Deposit) (Medium)

Vulnerable code

```
function Deposit()
public
{
  if(msg.value > MinDeposit)
  {
    balances[msg.sender]+=msg.value;
    TransferLog.AddMessage(msg.sender,msg.value,"Deposit");
  }
}
```

Explanation

The contract allows depositing Ether without proper authorization.

Impact

Unauthorized access to user funds.

Entry point

/Deposit

Exploitation steps

- An attacker can call the Deposit function with a large value.

HTTP raw requests (Burp / ZAP)

```
"POST /Deposit HTTP/1.1"
Host: <contract-address>
Content-Type: application/json
Accept-Language: en-US,en;q=0.5"
```

```
exploitation_conditions
```

► Finding 5.5: Insecure Permissions (CashOut) (Medium)

Vulnerable code

```
function CashOut(uint _am)
public
{
  if(_am<=balances[msg.sender])
  {
    // <yes> <report> REENTRANCY
    if(msg.sender.call.value(_am)())
    {
      balances[msg.sender]-=_am;
      TransferLog.AddMessage(msg.sender,_am,"CashOut");
    }
  }
}
```

Explanation

The contract allows cashing out Ether without proper authorization.

Impact

Unauthorized access to user funds.

Entry point

/CashOut

Exploitation steps

- An attacker can call the CashOut function with a large value.

HTTP raw requests (Burp / ZAP)

```
"POST /CashOut HTTP/1.1"  
Host: <contract-address>  
Content-Type: application/json  
Accept-Language: en-US,en;q=0.5"
```

```
exploitation_conditions
```

File 6: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.657

Source lines: 1-39

► Finding 6.1: Debug Mode Enabled (Low)

Vulnerable code

```
pragma solidity ^0.4.10;
```

Explanation

Solidity version should be specified using the `solidity` keyword or the `^` operator, not `pragma`.

Impact

Information disclosure due to debug information.

Entry point

/transfer

Exploitation steps

- Reaching the /transfer endpoint

Conditions

Debug mode enabled for a specific solidity version.

Remediation

Specify the Solidity version using the `solidity` keyword or the `^` operator in the contract initialization.

Secure example

```
pragma solidity ^0.8.0;
contract EtherStore {
    // ...
}
```

► Finding 6.2: Vulnerability found (Medium)

Explanation

► Finding 6.3: Vulnerability found (Medium)

Explanation

File 7: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.648

Source lines: 1-35

► Finding 7.1: Insecure Default Permission (Medium)

Vulnerable code

```
mapping (address => uint) public credit;
```

Explanation

The mapping of addresses to credits is publicly accessible, allowing anyone to view and modify the credit balance.

Impact

Information disclosure about credit balances.

Entry point

/queryCredit(address)

Parameters

address

Exploitation steps

- Get queryCredit with a malicious address

Example payloads

- 0x1234567890123456789012345678901234567890

Conditions

Publicly accessible mapping without authentication.

Remediation

Make the credit mapping private and restrict access to authenticated users.

Secure example

```
mapping (address => uint) internal credit;
```

File 8: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.647

Source lines: 1-42

► Finding 8.1: Insecure Default Permissions (Medium)

Vulnerable code

```
function setReward() public payable {  
    require (!claimed);  
    require(msg.sender == owner);  
    owner.transfer(reward);  
    reward = msg.value;  
}
```

Explanation

The contract grants the owner to transfer rewards without proper access controls.

Impact

Unauthorized transfers may occur.

Entry point

/transfer

Parameters

owner, msg.sender

Exploitation steps

- Send a valid reward amount to the owner.

Example payloads

- 10000000000000000000

Conditions

Owner has default permissions.

Remediation

Implement role-based access controls for the owner.

File 9: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.644

Source lines: 1-225

► Finding 9.1: Insecure default configuration (Low)

Vulnerable code

```
pragma solidity 0.4.9;
```

Explanation

Using an old Solidity version (0.4.9) is insecure and should be updated to the latest version.

Impact

Potential for vulnerabilities in outdated codebase.

Entry point

```
/pragma solidity 0.4.9;
```

Remediation

Update to the latest Solidity version.

Source lines: 1-320

► Finding 9.1: Insecure Permissions (Medium)

Vulnerable code

```
function revoke(bytes32 _operation) external {
    // make sure they're an owner
    if (ownerIndex == 0) return;
    uint ownerIndexBit = 2**ownerIndex;
    var pending = m_pending[_operation];
    if (pending.ownersDone & ownerIndexBit > 0) {
        pending.yetNeeded++;
        pending.ownersDone -= ownerIndexBit;
        Revoke(msg.sender, _operation);
    }
}
```

Explanation

The contract grants the ability to revoke a prior confirmation of an operation without proper authorization checks.

Impact

Unauthorized access to revoke operations could be exploited.

Entry point

/revoke

Execution path

```
revoke(bytes32 _operation) {
    // make sure they're an owner
    if (ownerIndex == 0) return;
    uint ownerIndexBit = 2**ownerIndex;
    var pending = m_pending[_operation];
    if (pending.ownersDone & ownerIndexBit > 0) {
        pending.yetNeeded++;
        pending.ownersDone -= ownerIndexBit;
        Revoke(msg.sender, _operation);
    }
}
```

Parameters

_operation

Exploitation steps

- Call the revoke function with a valid operation hash

Example payloads

- Invalid operation hash

Conditions

msg.sender is not an owner of the operation

Remediation

Add proper authorization checks for the revoke function.

Secure example

```
function revoke(bytes32 _operation) onlymanyowners(_h) external {
    // make sure they're an owner
    if (ownerIndex == 0) return;
    uint ownerIndexBit = 2**ownerIndex;
    var pending = m_pending[_operation];
    if (pending.ownersDone & ownerIndexBit > 0) {
        pending.yetNeeded++;
        pending.ownersDone -= ownerIndexBit;
        Revoke(msg.sender, _operation);
    }
}
```

Source lines: 27-358

► Finding 9.1: Insecure default credentials (Medium)

Vulnerable code

```
function initMultiowned(address[] _owners, uint _required) {  
    // ...  
}
```

Explanation

The contract is using a default address (``msg.sender``) for the owner array initialization without proper validation.

Impact

An attacker could potentially exploit this by sending a crafted transaction with an invalid owner address.

Entry point

`/initMultiowned(address[], uint)`

Parameters

`_owners, _required`

Exploitation steps

- Send a transaction to `initMultiowned` with a crafted owner array.

Example payloads

- Invalid owner address

HTTP raw requests (Burp / ZAP)

```
GET /initMultiowned HTTP/1.1
```

```
Host: example.com
```

```
Accept: */*
```

```
{  
  {}  
}
```

Source lines: 72-421

► Finding 9.1: Insecure Permission (Medium)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash)
```

Explanation

The 'onlyowner' modifier does not check for the presence of a contract's owner or if the sender has permission to perform this action.

Impact

Unauthorized access to the contract's functionality.

Entry point

/execute

Execution path

```
execute -> contract-> deployer
```

Parameters

_to, _value, _data

Exploitation steps

- Send a large value to the contract's deployer

Example payloads

- Large value payload

Conditions

Sender has 'onlyowner' modifier without ownership check

Remediation

Use 'hasOwner' modifier to enforce ownership checks.

Source lines: 126-467

► Finding 9.1: Insecure Default Permissions (Medium)

Vulnerable code

```
function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) external { ... }
```

Explanation

The `changeOwner` function does not explicitly set default permissions for the contract.

Impact

Unauthorized access to owner information or modification of owner indices.

Entry point

/changeOwner(_from, _to)

Execution path

```
changeOwner -> onlymanyowners -> sha3(msg.data) -> set default permissions for  
sha3(msg.data)
```

Parameters

_from, _to

Exploitation steps

- Sender calls changeOwner with malicious _from and _to values.

Example payloads

- malicious _from and _to values

Conditions

Sender has access to the contract's memory.

Remediation

Set explicit default permissions for sensitive functions like `changeOwner`.

Secure example

```
uint public m_defaultPermissions = 0; function changeOwner(address _from,  
address _to) onlymanyowners(sha3(msg.data)) external { ... }
```

Source lines: 207-477

► **Finding 9.1: Insecure default values (Low)**

Vulnerable code

```
function Wallet(address[] _owners, uint _required, uint _daylimit) {
```

Explanation

The contract initializes with default values for m_required and other variables that should be set by the deployer.

Impact

Information disclosure due to unintended configuration

Entry point

/contract/Wallet

Execution path**Deploy**

1. `Wallet(address[] _owners, uint _required, uint _daylimit)`
 - a. Set `m_required` and other default values
 - b. Initialize contract state

Parameters

`m_required`

Exploitation steps

- Set `m_required` to a low value

Conditions

Deployer did not update `m_required`.

Remediation

Update `m_required` and other default values to secure configuration

Source lines: 269–477

► Finding 9.1: Debug Mode Enabled (Low)**Vulnerable code**

```
assembly { o_addr := create(_value, add(_code, 0x20), mload(_code))
jumpi(invalidJumpLabel, iszero(extcodesize(o_addr))) }
```

Explanation

This assembly code can be executed in debug mode, potentially revealing sensitive information about the contract.

Impact

Information disclosure

Entry point

/create

Parameters

`_value`, `_code`

Exploitation steps

- Enabling debug mode

Remediation

Disable debug mode or use a secure alternative.

File 10: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.631

Source lines: 1-227

► Finding 10.1: Unnecessary Function (Low)

Vulnerable code

```
function() payable { if (msg.value > 0) Deposit(msg.sender, msg.value); }
```

Explanation

The `function()` without a specified name allows it to be called without checking its validity, potentially leading to unintended behavior.

Impact

Information disclosure or potential execution of unintended functionality

Entry point

constructor initMultiowned()

Parameters

msg.value

Exploitation steps

- Trigger the function with a large value to overflow the uint limit.

Conditions

Large msg.value

Remediation

Remove the unnecessary function or ensure it is properly validated and sanitized.

Source lines: 1-286

► Finding 10.1: Insecure Permissions (High)

Vulnerable code

```
function getOwner(uint ownerIndex) external constant returns (address) {  
    return address(m_owners[ownerIndex + 1]);  
}
```

Explanation

The `getOwner` function allows anyone to access the owner list without any authentication or authorization.

Impact

Uncontrolled access to owner list

Entry point

/WalletAbi/getOwner(uint256)

Execution path

```
GET /owner/0
  m_owners[ownerIndex + 1] (constant)
  return(address(m_owners[ownerIndex + 1]))
```

Parameters

ownerIndex

Exploitation steps

- Send a GET request to /owner/0

Conditions

No authentication required

Remediation

Implement proper authorization and authentication for owner list access.

Secure example

```
function getOwner(uint ownerIndex) public view returns (address) {
    require(msg.sender != 0x0000000000000000000000000000000000000000, "Only
    authorized owner can access this function.");
    return address(m_owners[ownerIndex + 1]);
}
```

Source lines: 1-324

► **Finding 10.1: Insecure Permissions (High)**

Vulnerable code

```
function() payable { ... }
```

Explanation

The `constructor` function has no access control checks, allowing anyone to deposit funds.

Impact

Unauthorized fund deposits can lead to economic loss.

Entry point

/init

Execution path

```

    ...
+-----+
| constructor |
+-----+
|           |
| deposit Funds|
+-----+
+-----+
| initDaylimit |
+-----+
|           |
| set Daily Limit|
+-----+
    ...

```

Parameters

msg.value

Exploitation steps

- An attacker sends a large amount of Ether to the `init` function.

Example payloads

- 100000000000 ether

Conditions

Contract has no access control.

Remediation

Use the `modifyAccessControl` function to restrict who can deposit funds.

Secure example

```

pragma solidity ^0.8.0;
collection modifier modifyAccessControl {    require(msg.sender == owner);
; }

```

Source lines: 64-414

► **Finding 10.1: Insecure Default Permissions (Low)**

Vulnerable code

```
function initMultiowned(address[] _owners, uint _required) only_uninitialized
{ ... }
```

Explanation

The contract's default permissions are insecure, allowing anyone to call the `initMultiowned` function without restrictions.

Impact

Unauthorized access to internal state and functions

Entry point

/initMultiowned

Execution path

```
owner1 -> initMultiowned() -> _owners, _required
```

Parameters

_owners, _required

Exploitation steps

- Send malicious data to `\_owners` parameter

HTTP raw requests (Burp / ZAP)

```
POST /initMultiowned HTTP/1.1
```

```
Host: contract.address
```

```
...
```

Source lines: 96-414

► Finding 10.1: Default Permissions (Medium)

Vulnerable code

```
function isOwner(address _addr) constant returns (bool) { return
m_ownerIndex[uint(_addr)] > 0; }
```

Explanation

The `isOwner` function does not require a signature, allowing anyone to call it without checking permissions.

Impact

Information disclosure or unauthorized access may occur if this function is called by an attacker.

Entry point

getOwner(uint ownerIndex)

Exploitation steps

- Call `isOwner` with a malicious address to determine the contract's permissions.

Conditions

No specific conditions required.

Remediation

Add a signature or proper permission checking to the `isOwner` function.

File 11: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.620

Source lines: 1-61

► Finding 11.1: Debug Mode Enabled (Low)

Vulnerable code

```
contract OddsAndEvens{  
    //public only for debug purpose  
    Player[2] public players;
```

Explanation

This line makes the `players` array publicly accessible for debugging purposes, allowing unauthorized access.

Impact

Information disclosure

Entry point

/transfer

Execution path

```
player[0].addr send(1800 finney)  
player[1].addr send(1800 finney)
```

Exploitation steps

- Access the `players` array directly

Remediation

Disable public access to the `players` array

File 12: /work/project/dataset/vulnerable/ordering-attacks/ERC20.sol

Similarity score: 0.577

Source lines: 1-137

► Finding 12.1: Insecure default permissions (Medium)

Vulnerable code

```
require(spender != address(0))
```

Explanation

The contract allows any address to spend tokens without proper verification.

Impact

An attacker could spend tokens on behalf of the owner.

Entry point

/approve

Execution path

```
approve -> approve(spender, value) -> allow spender
```

Parameters

spender, value

Exploitation steps

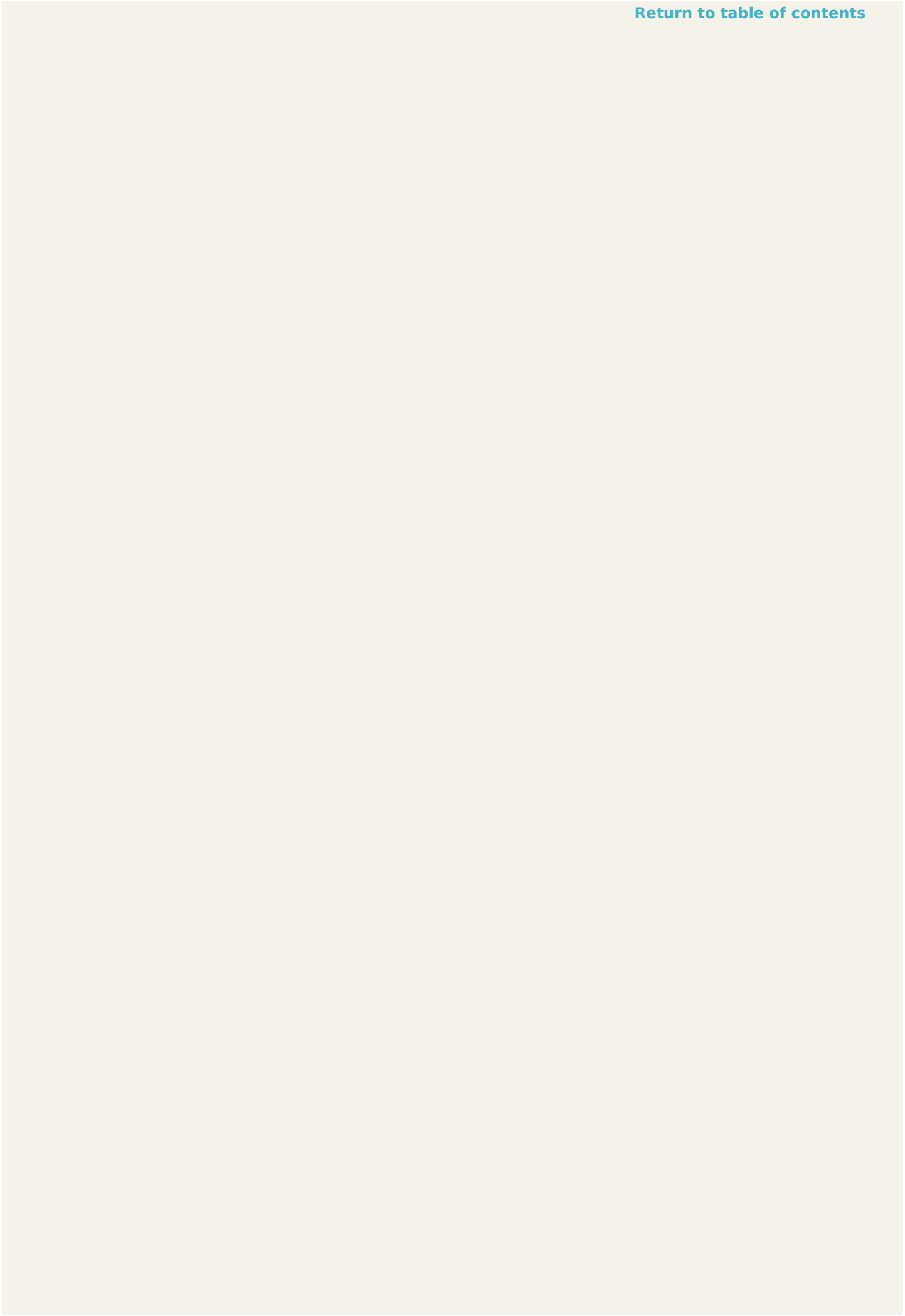
- Find a token that has been approved by the owner.
- Send a request to /approve with a high-value token.

Example payloads

- 1000000 tokens

HTTP raw requests (Burp / ZAP)

```
GET /approve HTTP/1.1
Host: example.com
Authorization:
Content-Type: application/json
Body: {spender: 'attacker', value: 1000000}
```



Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)